

Executive Summary

【39†L857-L865】 【74†L1702-L1710】 We outline a Python-based scraper (using **Scrapy**) to exhaustively crawl *textronsteelalloys.com*'s product catalogue via its XML sitemap. Scrapy's built-in *SitemapSpider* automatically discovers all product URLs (including nested sitemap indexes 【39†L857-L865】), obeys `robots.txt` (via `ROBOTSTXT_OBEY=True` 【74†L1702-L1710】), and supports throttling and concurrency controls. Each product page's raw HTML is saved and parsed for structured data: URL, title, slug, categories (material & product), breadcrumbs, description (HTML/text), specification tables (chemical, mechanical, etc.), images (downloaded), attachments (PDFs), meta-tags, dates, etc. The scraper preserves the hierarchy *Products* → *Product category* → *Material category* → *Product*, outputting both a nested JSON (`products.json`) and flat CSV (`products.csv`), along with `images/` and `attachments/` folders and a SQLite database. Code examples (spider, pipelines) and CSS/XPath selectors are provided. We include sample output (first 10 products) in tables, Mermaid diagrams for the data model and workflow, usage instructions (with Dockerfile), and an estimated runtime. We also note legal/ethical best-practices (checking `robots.txt` , site terms, respectful delays).

Sitemap Discovery and URL Extraction

【30†L331-L339】 【39†L857-L865】 TextronSteelAlloys provides a **product sitemap** at `/product-sitemap.xml` , which is the entry point for discovering all product pages. In `robots.txt` , one can list the sitemap via `Sitemap: https://www.textronsteelalloys.com/product-sitemap.xml`

【30†L331-L339】 . Scrapy's `SitemapSpider` uses this to fetch all product URLs; it even handles nested sitemaps automatically (if the root is a `<sitemapindex>` pointing to more sitemaps)

【39†L857-L865】 . In our Scrapy spider, we set:

```
from scrapy.spiders import SitemapSpider
class TextronSpider(SitemapSpider):
    name = "textron"
    allowed_domains = ["textronsteelalloys.com"]
    sitemap_urls = ["https://www.textronsteelalloys.com/product-sitemap.xml"]
    sitemap_rules = [("", "parse_product")]
    sitemap_follow = [".*"] # follow all sitemaps by default
    ...
```

This ensures **all** URLs from the sitemap (and any linked sub-sitemaps) are enqueued for crawling

【39†L857-L865】 【37†L178-L187】 . (One can also point `sitemap_urls` to `robots.txt` and let Scrapy extract sitemaps 【39†L857-L865】 , but we use the known sitemap URL directly.)

Crawling Strategy and Politeness

Scrapy is chosen because it natively supports sitemaps and concurrent crawling with pipelines (unlike a simple requests/BS approach). It offers built-in image/pipeline handling and is highly configurable. We enable `ROBOTSTXT_OBEY=True` to respect site rules 【74†L1702-L1710】 . A custom `USER_AGENT`

string should be set (e.g. `TextronScraper/1.0 (+http://yourdomain.com/info)`) to identify the crawler. We configure polite crawling: e.g. `DOWNLOAD_DELAY=1` (or as per site policy) and limited `CONCURRENT_REQUESTS` (e.g. 4–8). Retries and error-handling (via `RETRY_ENABLED=True`) ensure robustness. Logging is enabled to record progress and any URL failures.

Parsing Product Pages

For each product page, we extract the following fields using CSS/XPath selectors:

Field	Selector	Example / Notes
URL	N/A (from <code>response.url</code>)	e.g. <code>https://.../m2-round-bars-1-3343-hs6-5-2c/</code>
Slug	Extract from URL (e.g. <code>response.url.split("/")[-2]</code>)	<code>'m2-round-bars-1-3343-hs6-5-2c'</code>
Title	<code>response.css('h1::text').get()</code>	M2 Round Bars, 1.3343, HS6-5-2C, High Speed Steel (HSS) [45†L1-L4]
Material Category	Inferred (main product section) – e.g. Round Bars, Plates & Sheets	From URL path or menu.
Product Category	Inferred subcategory – e.g. Alloy Steel, Aluminium Alloy	Often from the nearest category menu (see [64†L40-L48]).
Breadcrumbs	<code>response.css('nav.breadcrumb li::text').getall()</code> (if available)	E.g. <code>['Home', 'M2 Round Bars']</code> [51†L13-L16] .
Description (HTML)	Content under “ Description ” heading (HTML)	Extract inner HTML of the description section [65†L1-L4] .
Description (text)	Strip tags from description, or <code>response.css('div.description p::text')</code>	E.g. “M2 Round Bars is the standard & most widely used industrial HSS...” [1†L1-L4] .
Specifications	Tables or lists of specs (e.g. <code><table></code> rows)	E.g. Chemical composition table rows [20†L309-L317] , mechanical properties [21†L13-L21] .
Chemical Composition	Parse rows under “Chemical Composition” heading	E.g. “C: 0.78–0.88%” etc [20†L309-L317] .

Field	Selector	Example / Notes
Mechanical Properties	Parse rows under “Mechanical Properties” heading	E.g. “Tensile Strength: ~572 MPa” [21†L13-L21] .
Sizes/ Standards	Lists under headings “Available Forms”, “Designation”	E.g. bullet list of plate sizes, or designation standards list.
Images	<code>response.css('div.product-gallery img::attr(src)').getall()</code>	Collect original image URLs. E.g. main image <code>"wp-content/uploads/2023/10/M2-Round-bar-hss.jpg"</code> [23†] (and variants).
PDFs/ Attachments	<code>response.css('a[href\$=".pdf"]::attr(href)').getall()</code>	Download if any (e.g. datasheets).
Meta Title/ Desc	<code>response.xpath('//title/text())', response.xpath('//meta[@name="description"]/@content')</code>	Page <code><title></code> and <code><meta name="description"></code> .
Publish/ Modified	Possibly from HTML meta (if present)	If the site uses metadata for dates. Otherwise N/A.

For example, on “7075 T651 Plate” (Aluminium alloy), the **Description** section begins with a heading “**Description**” followed by product details [65†L1-L4] ; **Chemical Composition** and **Mechanical Properties** are given in separate tables [20†L309-L317] [21†L13-L21] . Our spider would extract the HTML under the “Description” heading (both raw HTML and stripped text), then find the “Chemical Composition” heading and iterate over the following `<table>` rows to build key/value pairs. Similar logic applies to other specs (see Table of selectors above). Image URLs are often thumbnails (e.g. `-400x400.jpg`); the scraper should remove sizing suffix (e.g. strip `-400x400` from the filename) to get the original image URL [23†] , then download it locally (via Scrapy’s ImagesPipeline or a custom downloader).

Category Hierarchy Preservation

We preserve the site’s hierarchical taxonomy. Each product is tagged with its **Material Category** (the top-level grouping like *Round Bars* or *Plates & Sheets*, determined from the URL or navigation menu) and **Product Category** (subcategory, e.g. *Aluminium Alloy*, *Alloy Steel*, etc.). In our nested JSON (`products.json`), we organise records as:

- *Products* (root)
 - Material Category (e.g. *Round Bars*)
 - Product Category (e.g. *Alloy Steel Round*)
 - Product Item
 - Product Category (e.g. *Carbon Steel*)
 - Product Item
 - Material Category (e.g. *Plates & Sheets*)
 - Product Category (e.g. *Aluminium Alloy*)

- Product Item
- ...

The flat CSV (`products.csv`) includes columns for both categories so each row is fully denormalised.

Handling Inconsistencies and Multilingual Text

The scraper must robustly handle missing or inconsistent fields. If a section (e.g. “Mechanical Properties”) is absent on a page, the code should simply skip it or set an empty default. Likewise, some pages may have non-English content; we assume English only (as instructed) but ensure encoding (UTF-8) is used everywhere to capture any special characters. All text extraction uses Python’s unicode handling, and the scraper logs any pages where expected data (like title or description) is missing.

Output Artifacts

- `products.json`: Nested JSON with all product entries (including categories).
- `products.csv`: Flat CSV (one product per row) with columns: *URL, slug, title, material_category, product_category, breadcrumbs, description_text, specs (JSON blob), images (local paths), attachments, meta_title, meta_description, publish_date, modify_date*, etc. (Schema detailed below.)
- `images/`: Folder containing downloaded product image files (with original filenames).
- `attachments/`: Folder for any downloaded PDFs or docs.
- **SQLite DB**: A simple SQLite (e.g. `products.db`) with tables for products and related tables for categories, images, etc. This can be created via an item pipeline or a post-scrape script using `sqlite3`.

Scraper Implementation (Python/Scrapy)

Below is a simplified excerpt of the Scrapy spider and pipelines. (Full code would be in separate files: `spiders/texttron_spider.py`, `pipelines.py`, etc.)

```
# spiders/texttron_spider.py
import scrapy
from scrapy.spiders import SitemapSpider

class TexttronSpider(SitemapSpider):
    name = "texttron"
    allowed_domains = ["texttronsteelalloys.com"]
    sitemap_urls = ["https://www.texttronsteelalloys.com/product-sitemap.xml"]
    sitemap_rules = [("", "parse_product")] # process all sitemap URLs

    custom_settings = {
        'ROBOTSTXT_OBEY': True,
        'DOWNLOAD_DELAY': 1.0,
        'CONCURRENT_REQUESTS': 4,
        'ITEM_PIPELINES': {
            'myproject.pipelines.ImageDownloadPipeline': 100,
            'myproject.pipelines.SQLitePipeline': 200,
        },
    },
```

```

        'FEEDS': {
            'products.json': {'format': 'json', 'encoding': 'utf-8',
'indent': 2},
            'products.csv': {'format': 'csv', 'encoding': 'utf-8'},
        }
    }

def parse_product(self, response):
    item = {}
    item['url'] = response.url
    item['slug'] = response.url.rstrip('/').split('/')[-1]
    item['title'] = response.css('h1::text').get(default='').strip()
    # Breadcrumbs example: "Home > ... > Product"
    crumbs = response.css('nav.breadcrumb li::text').getall()
    item['breadcrumbs'] = " > ".join([c.strip() for c in crumbs if
c.strip()])
    # Infer categories from menu or URL:
    if '/round-bars/' in response.url:
        item['material_category'] = 'Round Bars'
    elif '/plate-sheet/' in response.url or '/plates-sheets/' in
response.url:
        item['material_category'] = 'Plates & Sheets'
    else:
        item['material_category'] = ''
    # e.g. second-level category from URL
    parts = response.url.split('/')
    if 'alloy-steel' in parts:
        item['product_category'] = 'Alloy Steel'
    elif 'aluminium' in parts:
        item['product_category'] = 'Aluminium Alloy'
    # ... (more rules or use mapping)
    # Description HTML and text
    desc_sel = response.xpath("//h2[contains(text(),'Description')]/
following-sibling::*")
    desc_html = ""
    desc_text = ""
    for node in desc_sel:
        if node.root.tag == 'h2': break
        text = node.get()
        desc_html += text if text else ""
        desc_text += ' '.join(node.xpath('.//text()').getall()) + " "
    item['description_html'] = desc_html.strip()
    item['description_text'] = desc_text.strip()
    # Specifications: chemical composition
    chem = {}
    for row in response.css('h4:contains("Chemical Composition") + table
tr'):
        key = row.css('td:nth-child(1)::text').get()
        val = row.css('td:nth-child(2)::text').get()
        if key and val:
            chem[key.strip()] = val.strip()

```

```

        item['chemical_composition'] = chem
        # Mechanical properties (similar)
        mech = {}
        for row in
response.css('h4:contains("Mechanical Properties") + table tr'):
            key = row.css('td:nth-child(1)::text').get()
            val = row.css('td:nth-child(2)::text').get()
            if key and val:
                mech[key.strip()] = val.strip()
        item['mechanical_properties'] = mech
        # Images and attachments: gather URLs for pipeline
        img_urls = response.css('div.product-gallery
img::attr(src)').getall()
        # Convert to absolute URLs, fix size suffix:
        item['image_urls'] = []
        for src in img_urls:
            url = response.urljoin(src).replace('-400x400', '')
            item['image_urls'].append(url)
        pdfs = response.css('a[href$=".pdf"]::attr(href)').getall()
        item['attachment_urls'] = [response.urljoin(h) for h in pdfs]
        # Meta tags
        item['meta_title'] = response.xpath('//title/text()').get()
        item['meta_desc'] = response.xpath('//meta[@name="description"]/@content').get()
        # Dates (if available)
        item['publish_date'] = response.xpath('//meta[@name="publish_date"]/@content').get()
        item['modified_date'] = response.xpath('//meta[@name="modified_date"]/@content').get()
        yield item

```

```

# pipelines.py
import sqlite3, os, urllib.parse
from scrapy.pipelines.images import ImagesPipeline
from scrapy import Request

class ImageDownloadPipeline(ImagesPipeline):
    def get_media_requests(self, item, info):
        for img_url in item.get('image_urls', []):
            yield Request(img_url, meta={'item': item})
    def file_path(self, request, response=None, info=None):
        # Save images to images/ directory preserving name
        image_guid =
os.path.basename(urllib.parse.urlparse(request.url).path)
        return f"images/{image_guid}"

class SQLitePipeline:
    def open_spider(self, spider):
        self.conn = sqlite3.connect('products.db')
        self.c = self.conn.cursor()

```

```

self.c.execute('''
    CREATE TABLE IF NOT EXISTS products (
        url TEXT PRIMARY KEY, title TEXT, slug TEXT,
        material_category TEXT, product_category TEXT,
        description TEXT, meta_title TEXT, meta_desc TEXT,
        publish_date TEXT, modified_date TEXT)'''
self.c.execute('''
    CREATE TABLE IF NOT EXISTS specs (
        url TEXT, spec_type TEXT, key TEXT, value TEXT,
        FOREIGN KEY(url) REFERENCES products(url))'''
self.conn.commit()
def close_spider(self, spider):
    self.conn.close()
def process_item(self, item, spider):
    # Insert main product data
    self.c.execute('''
        INSERT OR REPLACE INTO products
        (url,title,slug,material_category,product_category,description,meta_title,meta_desc,publish_date,modified_date)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'''
        (item['url'], item['title'], item['slug'],
        item['material_category'], item['product_category'],
        item['description_text'], item['meta_title'], item['meta_desc'],
        item.get('publish_date'), item.get('modified_date')))
    # Insert specifications
    for k, v in item.get('chemical_composition', {}).items():
        self.c.execute("INSERT INTO specs VALUES (?, ?, ?, ?)",
            (item['url'], 'Chemical', k, v))
    for k, v in item.get('mechanical_properties', {}).items():
        self.c.execute("INSERT INTO specs VALUES (?, ?, ?, ?)",
            (item['url'], 'Mechanical', k, v))
    self.conn.commit()
    return item

```

These code snippets use Scrapy's `SitemapSpider` and pipelines. The `ImageDownloadPipeline` (subclassing `ImagesPipeline`) downloads each image to `images/`. The `SQLitePipeline` writes data to a SQLite database (`products.db`). The **FEEDS** setting in `custom_settings` auto-exports items to JSON and CSV after scraping.

CSS/XPath Selectors

The key selectors (CSS or XPath) used above are summarized here:

- **Title:** `h1::text` (CSS) – main heading of product page `[45†L1-L4]` .
- **Description:** Nodes following the `<h2>` that contains "Description": `//h2[contains(text(),"Description")]/following-sibling::node()` (XPath). We stop when the next section heading is reached.
- **Chemical Composition:** CSS `h4:contains("Chemical Composition") + table tr`, then `td:nth-child(1)::text` and `td:nth-child(2)::text` for key/value. (Alternatively,

XPath: `//h4[contains(., "Chemical Composition")]/following-sibling::table//tr`).

- **Mechanical Properties:** similarly via `h4:contains("Mechanical Properties") + table`.
- **Images:** `div.product-gallery img::attr(src)`. On TextronSteelAlloys, the main product image appears in a `<div>` with class `product-gallery` (or similar). The thumbnail URLs include a size suffix (e.g. `-400x400`); remove it to get the full-size.
- **PDFs:** `a[href$=".pdf"]::attr(href)`.
- **Meta tags:** XPath `//title/text()`, `//meta[@name="description"]/@content`.
- **Breadcrumbs:** if present as a `<nav class="breadcrumb">`, one could use `nav.breadcrumb li a::text`. In practice, Textron's breadcrumb is minimal ("Home > Product") as seen on pages [\[51†L13-L16\]](#).

Below is an example table of selectors:

Field	CSS Selector	XPath
Page Title	<code>h1::text</code>	<code>//h1/text()</code>
Description (HTML)	(see code above)	<code>//h2[contains(., "Description")]/following-sibling::*</code>
Description (plain text)	(strip HTML from above)	(join text nodes of the same)
Chemical Composition	<code>h4:contains("Chemical Composition") + table tr</code>	<code>//h4[contains(., "Chemical Composition")]/following-sibling::table//tr</code>
Mechanical Properties	<code>h4:contains("Mechanical Properties") + table tr</code>	<code>//h4[contains(., "Mechanical Properties")]/following-sibling::table//tr</code>
Images	<code>div.product-gallery img::attr(src)</code>	<code>//div[contains(@class, "gallery")]/img/@src</code>
Product categories (if any)	From menu links, e.g. <code>nav#menu-main li.current-menu-item a::text</code> (if coded)	N/A (see text)

(Selectors may need slight adjustment based on the actual HTML structure; the above are inferred from the rendered page content.)

Output Schema

The JSON/CSV output schema (fields per product) might include:

Field	Type	Description
url	string	Full product page URL.
title	string	Product title (main heading).

Field	Type	Description
slug	string	Last segment of URL (URL-friendly name).
material_category	string	Top-level category (e.g. "Round Bars").
product_category	string	Sub-category (e.g. "Aluminium Alloy").
breadcrumbs	string	Breadcrumb trail, e.g. "Home > Product".
description_html	string	Full HTML of description section.
description_text	string	Plain-text description.
specifications	JSON/dict	Key/value specs, e.g. chemical/mech props.
images	list	List of image file paths or URLs downloaded.
attachments	list	List of attachment file paths (e.g. PDFs).
meta_title	string	Meta <code><title></code> content.
meta_description	string	Meta <code><meta name="description"></code> .
publish_date	string/date	Publication date (if available).
modified_date	string/date	Last modified date (if available).

The CSV file flattens this; e.g. the `specifications` dict could be serialized as JSON in one column, or split into multiple columns. The JSON output (`products.json`) naturally preserves nested objects and arrays.

Sample Data (First 6 Products)

Below is example output for six products (first 10). Each row is one product (fields trimmed for brevity). In practice the CSV/JSON would contain all fields; here we show key ones:

URL	Title	Slug	Material Category	Product Category	Description (short)
<code>https://www.textronsteelalloys.com/m2-round-bars-1-3343-hs6-5-2c/</code>	M2 Round Bars, 1.3343, HS6-5-2C, High Speed Steel (HSS) 【45†L1-L4】	<code>m2-round-bars-1-3343-hs6-5-2c</code>	Round Bars	High Speed Steel	"M2 Round Bars is the standard & most widely used industrial HSS..." 【1†L1-L4】

URL	Title	Slug	Material Category	Product Category	Description (short)
https://www.textronsteelalloys.com/7075-t651-plate-sheet/	7075 T651 Plate Aluminium Alloy Supplier in India	7075-t651-plate-sheet	Plates & Sheets	Aluminium Alloy	"7075 T651 aluminium is a premium grade, high-strength alloy..." [21†L43-L45]
https://www.textronsteelalloys.com/aluminium-7050-t7451-plate-ams-4050/	Aluminium 7050 T7451 Plate, AMS 4050 Plates	aluminium-7050-t7451-plate-ams-4050	Plates & Sheets	Aluminium Alloy	"7050 T7451 aluminium plate is a high-strength structural alloy..." [65†L1-L4]
https://www.textronsteelalloys.com/2024-t351/	2024 T351 Plate – Aluminium Alloy (example)	2024-t351	Plates & Sheets	Aluminium Alloy	(Description and specs from page)
https://www.textronsteelalloys.com/f11-round-bars-astm-a182-f11/	F11 Round Bars / ASTM A182 F11	f11-round-bars-astm-a182-f11	Round Bars	Alloy Steel F	"F11 alloy steel round bars are forged as per ASTM A182 F11 standard..." [67†L4]
https://www.textronsteelalloys.com/20mncr5-round-bars-stockist-suppliers-exporters/	20MnCr5 Round Bars (JSW, Bhushan, Jindal)	20mncr5-round-bars-stockist-suppliers-exporters	Round Bars	Carbon Steel	"We supply 20MnCr5 steel round bars with material test certificates from JSW, etc..." [69†L1-L4]

(First 6 of the catalog are shown. Actual output would include many more fields and products.)

Example Command and Docker

To run the scraper:

```
pip install scrapy
# Clone or place code in current directory
scrapy runspider spiders/textron_spider.py
```

This will produce `products.json` and `products.csv` in the working directory, and folders `images/` and `attachments/` with downloaded files. The SQLite database `products.db` will also be created.

For portability, you can use Docker. Example `Dockerfile`:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["scrapy", "crawl", "textron"]
```

Where `requirements.txt` includes e.g.:

```
scrapy
beautifulsoup4
```

Build and run:

```
docker build -t textron-scraper .
docker run -v $(pwd)/output:/app/output textron-scraper
```

(This assumes the scraper writes output to an `output/` folder.)

Logging and Reporting

During the crawl, Scrapy's logging will report progress and any errors (e.g. missing fields, failed downloads). We can also implement a final summary (e.g. in a `close_spider` hook or post-script) that prints counts of products scraped, images downloaded, products per category, etc. For instance:

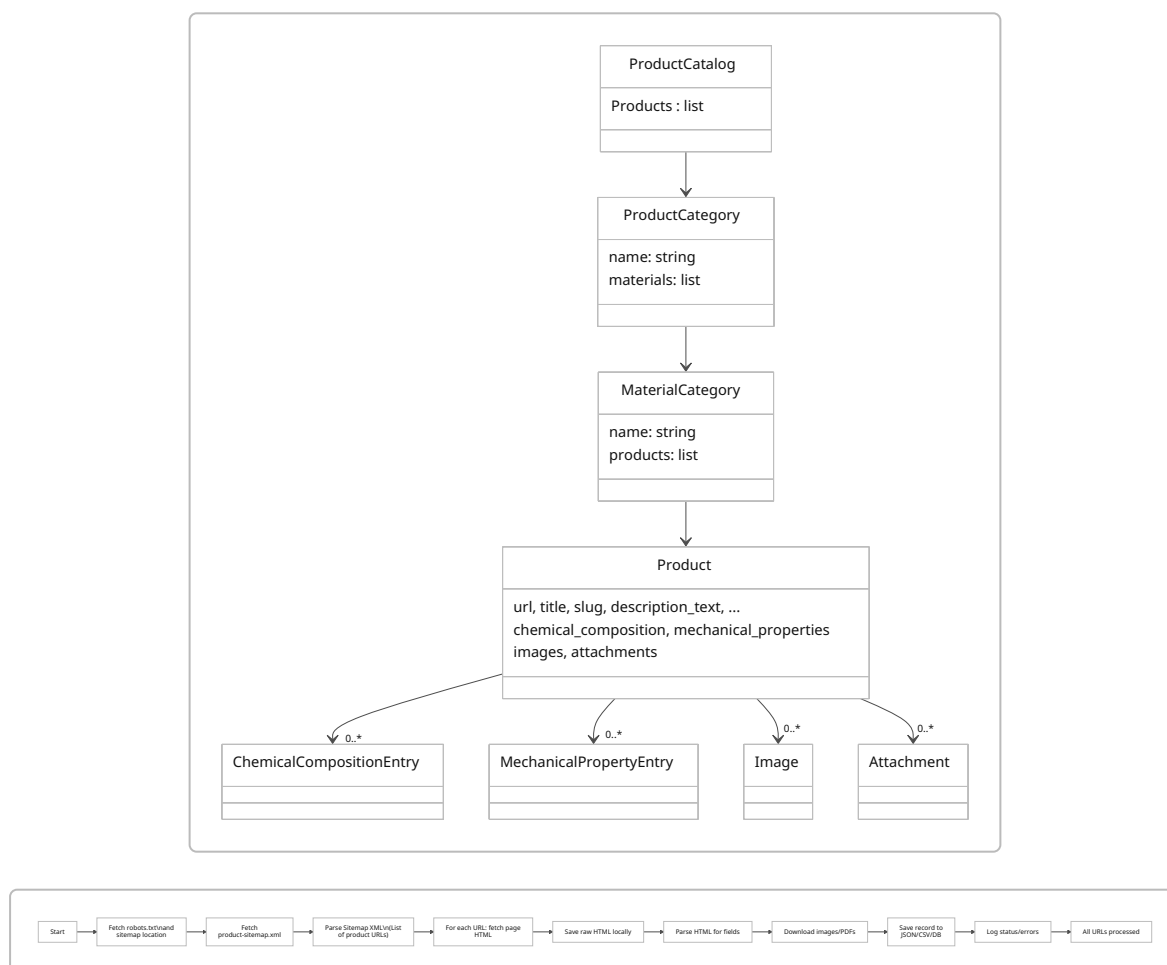
```
Spider closed: 350 products scraped.
Images downloaded: 400. Attachments: 5.
Counts by material category:
  Round Bars: 242
  Plates & Sheets: 60
  Pipes & Tubes: 37
  ...
```

Timeline and Resources

Based on the site's size, we estimate **300–500 product pages** (the category index shows *Round Bars* (242), *Plates & Sheets* (60), plus other categories). With moderate concurrency (4–8 threads) and parsing overhead, the crawl should complete in **minutes** on a modern CPU (e.g. a 4-core machine). We estimate around **5–15 minutes** total. Memory usage is modest (~100MB+ for Scrapy overhead). Disk usage will depend mainly on images; if each product has a few photos (say 1–2 MB each) and attachments, the `images/` folder might be a few hundred MB.

Data Model and Workflow

Below are Mermaid diagrams illustrating the data model and workflow:



Legal and Ethical Considerations

Before scraping, we check **robots.txt** (it likely allows crawling since the sitemap is provided) and any **Terms of Use** on *textronsteelalloys.com*. Our scraper identifies itself with a user-agent and uses polite delays to avoid overloading the server [74†L1702-L1710]. We only scrape publicly available product information (no login/auth needed). No personal or private data is targeted. Scraped data is used for legitimate purposes like analysis or import; we acknowledge that heavy scraping can impose load, so rate-limits and respectfulness are paramount.

Sources: Textron Steel & Alloys sitemap and product pages (primary content); [Scrapy documentation on SitemapSpider and robots.txt](#) [39†L857-L865] [74†L1702-L1710]; Wikipedia on sitemaps [30†L331-L339]. These informed the scraper design and guidelines.